

SpookyProxy

Description: I just learned how to build a proxy. Hope nothing is wrong with my application!

Author: RJCyber

Solution

The challenge runs a simple Flask application that has pages to login and register a user. The goal is to become the admin user to read the flag from the dashboard. As the challenge name and description hint at, there is a proxy (specifically, an nginx proxy). Note that although competitors may consider SQL injection, it will not be possible because all DB queries are parameterized in `app.py`.

The vulnerability in this application lies in the `proxy.conf` file. Here is the vulnerable aspect of the nginx config:

```
location /static {  
    alias /var/www/static/;  
    expires 1d;  
    add_header Cache-Control "public, immutable";  
}
```

Since there is no trailing slash at the end of the `/static` location, it introduces a Local File Inclusion (LFI) vulnerability. This is because competitors could append `../` after `/static` to access other paths in the webroot (which is set to `/var/www/html` by the Dockerfile). However, the `flag.txt` file is not accessible since the Dockerfile moves it to `/`.

The next part of the challenge lies in understanding the directory structure of the service. Since the alias for `/static` is set to `/var/www/static/`, we can go back a directory and then navigate to `html` to read the challenge source. This is important because we can then read the admin's password from `setup.py` by navigating to `/static../html/setup.py` (this is because Dockerfile copies `setup.py` to the webroot).

The (unredacted) `setup.py` script has the following contents:

```
import sqlite3  
from werkzeug.security import generate_password_hash  
  
DATABASE = 'challenge.db'  
  
FLAG = open('flag.txt').read().strip()
```

```

def setup():
    conn = sqlite3.connect(DATABASE)
    cursor = conn.cursor()

    cursor.execute('''CREATE TABLE IF NOT EXISTS users (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        username TEXT UNIQUE NOT NULL,
                        password TEXT NOT NULL
                    )''')

    cursor.execute('''CREATE TABLE IF NOT EXISTS flags (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        flag TEXT NOT NULL
                    )''')

    try:
        cursor.execute("INSERT INTO users (username, password) VALUES (?, ?)",
                        ('admin', generate_password_hash('Uncr4ck4bl3P4ss!!!')))
        cursor.execute("INSERT INTO flags (flag) VALUES (?)", (FLAG,))
        conn.commit()
        print("Database setup complete.")
    except sqlite3.IntegrityError:
        print("Admin user or flag already exists.")

    conn.close()

if __name__ == '__main__':
    setup()

```

Another way to go about utilizing the LFI vulnerability would be to consider the fact that the setup script creates a database file called `challenge.db`. This file can also be navigated to in the webroot by accessing `/static../html/challenge.db`. Since the `setup.py` script writes the flag to the `flags` table, the plaintext flag can be found in the database file.

Flag: `SVBGR{wh00ps_1_f0rg0t_4_sl4sh_1n_my_ng1nx_c0nf1g}`